Table of Contents

- **1. Executive Summary & Abstract**
- **2. Introduction & Local Context**
  - 2.1 Public Transport Challenges in Sierra Leone
  - 2.2 Link to UN Sustainable Development Goal 11 (SDG 11)
- **3. Problem Decomposition & Algorithmic Design**
  - 3.1 Breaking Down the Problem
  - 3.2 Abstraction
- **4. System Logic & Input Validation**
  - 4.1 Input-Process-Output (IPO) Model
  - 4.2 Algorithm & Pseudocode with Input Validation
- **5. Flowchart & Design Structure**
  - 5.1 Flowchart Symbols and Structure
- **6. Design Rationale**
  - 6.1 Logic & Validation Rationale
  - 6.2 Flowchart & Layout Rationale
- **7. Open Source Software & Digital Public Goods**
  - 7.1 Open Source Principles
  - 7.2 Version Control & GitHub Deliverables
- **8. Conclusion & Reflection**
- **9. References**

1. Executive Summary & Abstract

When I started this assignment, my goal was to look at a real everyday issue we experience here in Freetown, unpredictable public transport fares and design a simple logic algorithm to fix it. Commuters taking poda-podas or shared taxis along busy routes like Kissy Road, Lumley, or Regent often face sudden price increases during heavy traffic or peak rush hours.

To solve this, I created a step-by-step logical design using pseudocode and a flowchart diagram. My algorithm takes inputs like distance zones, passenger counts, and peak traffic hours, and calculates a fair transport-fare in Sierra Leone . Crucially, I built in explicit input validation checks so that if someone types an invalid zone (outside zones 1 to 4) or tries to overload a vehicle past its seating capacity, the system rejects it immediately with an error message instead of assigning a wrong fare. I also uploaded my work step-by-step to a public GitHub repository to follow Open Source and Digital Public Goods standards.

3

2. Introduction & Local Context

2.1 Public Transport Challenges in Sierra Leone

In Sierra Leone, especially in Freetown and surrounding towns, almost everyone relies on commercial minibuses (poda-podas) and shared taxis to get to school or work. However, because there isn't a digital or standardized pricing system, passengers and conductors often end up arguing over charges. Fares tend to change randomly depending on fuel costs, sudden rainstorms, or heavy traffic jams around major junctions like Congo Cross or Eastern Police.

By applying what I learned in PROG101, I wanted to design a clear, transparent algorithm that calculates transport prices automatically based on set rules. Having an automated system protects passengers from being overcharged and helps transport operators keep clean, honest boarding records.

2.2 Link to UN Sustainable Development Goal 11 (SDG 11)

UN SDG 11 focuses on making cities safe, inclusive, resilient, and sustainable. Target 11.2 specifically calls for affordable and accessible public transport systems for everyone. My project directly aligns with this goal because making transport fare logic transparent and publicly accessible helps build fairer urban transit in local communities.

3. Breaking Down the Problem

When looking at the transport system, trying to solve everything at once felt overwhelming. So, I used problem decomposition to break the system into four smaller, manageable steps:

1. **Input Collection & Validation:** Asking for the distance zone, maximum vehicle capacity, total passengers boarding, and peak hour status and making sure the numbers are valid before proceeding.
2. **Base Fare Calculation:** Assigning a specific base fare depending on the distance zone selected (Zone 1 through Zone 4).
3. **Surcharge Application:** Checking if peak-hour travel conditions apply and adding a percentage surcharge if true.
4. **Displaying Results:** Printing a clear summary showing the validated booking and total cost in Sierra Leone.

3.1 Abstraction

To keep my logic focused, I used abstraction to filter out unnecessary details. For example, I didn't include vehicle engine types, driver names, or fuel brand prices in the code. I focused strictly on the core variables that directly affect ticket pricing: distance zone, peak travel time, vehicle seating capacity, and passenger count.

4. System Logic & Input Validation

4.1 Input-Process-Output (IPO) Model

I structured my system using the IPO model to keep the logical flow clear:

Input	Processing	Output
• Distance Zone (1 to 4) • Vehicle Capacity • Passenger Count • Peak Hour Flag (Yes/No)	1. Validate Zone (Must be 1, 2, 3, or 4) 2. Validate Capacity (> 0) 3. Validate Passengers (> 0 and <= Capacity) 4. Calculate base fare from zone 5. Apply 20% peak surcharge if applicable	• Booking Confirmation • Validated Passenger Count • Final Fare (in Leones)

4.2 Algorithm & Pseudocode with Input Validation

Here is the complete pseudocode I wrote, including the validation loops to reject invalid distance zones and vehicle overcrowding:

```
MainProgram:
    Start
    Declare zone, passengers, capacity As Integer
   Declare isPeak As String
    Declare fare As Real
    // Validation 1: Distance Zone Input (Zones 1-4)
    Display "Enter Distance Zone (1: Short, 2: Medium, 3: Long, 4: Express):"
    Input zone
    While (zone < 1 OR zone > 4)
        Display "ERROR: Invalid distance zone (1-4). Re-enter:"
        Input zone
    EndWhile
    // Validation 2: Vehicle Capacity Input
    Display "Enter Maximum Vehicle Seating Capacity:"
    Input capacity
    While (capacity <= 0)
        Display "ERROR: Capacity must be > 0. Re-enter:"
        Input capacity
    EndWhile
    // Validation 3: Passenger Count Input
    Display "Enter Number of Passengers Boarding:"
    Input passengers
    While (passengers <= 0 OR passengers > capacity)
        If passengers > capacity Then
            Display "ERROR: Overcrowding detected!"
        Else
            Display "ERROR: Invalid passenger count."
        EndIf
        Display "Re-enter valid passenger count:"
        Input passengers
    EndWhile
    Display "Is it peak hour? (Yes/No):"
    Input isPeak
    fare = CalculateFare(zone, isPeak)
    Display "========================================="
    Display "TRANSIT TICKET SUMMARY"
    Display "Zone: " + zone + " | Passengers: " + passengers + " / " + capacity
    Display "Total Fare: Le " + fare
    Display "========================================="
    End
Function CalculateFare(zone, peak) As Real
    Declare baseFare, totalFare As Real
    If zone == 1 Then
        baseFare = 5.0
    ElseIf zone == 2 Then
        baseFare = 8.0
    ElseIf zone == 3 Then
        baseFare = 12.0
    ElseIf zone == 4 Then
        baseFare = 18.0
    EndIf
    If peak == "Yes" OR peak == "yes" Then
        TotalFare = BaseFare * 1.20
    Else
        TotalFare = BaseFare
    EndIf
    Return totalFare
EndFunction
```

5. Flowchart And Design Structure

5.1 Flowchart Logic Structure

- Start (Oval): Initialize system variables for route ID, capacity, and passenger counts.
- Input (Parallelogram): Enter RouteID, MaxCapacity, and Initial PassengerCount.
- Decision (Diamond): Is PassengerCount < MaxCapacity?
    - No: Output "Vehicle Full: Boarding Closed" and terminate process.
    - Yes: Proceed to input Distance Zone and Peak Hour status.
- Processing (Rectangle): Call CalculateFare function, compute base fare with peak surcharge, and increment passenger count by 1.
- Output (Parallelogram): Display ticket summary and updated passenger manifest.

5.2 Functional Modules & Sample Outputs

- Scenario Input: Route ID: "FRT-04", Distance Zone: 2 (Medium), Peak Hour: "YES", Initial Passengers: 13, Max Capacity: 14.
- Processed Output Calculation: Base fare for Zone 2 is Le 10.00, adjusted by a 1.25 peak multiplier resulting in a final fare of Le 12.50. Passenger count updates from 13 to 14.
- System Display Messages:
  Boarded Successfully. Fare: Le 12.50
  Current Count: 14 / 14
  WARNING: Vehicle Full. Boarding Closed.

6. Design Rationale

6.1 Logic & Validation Rationale

To guarantee system accuracy, I designed the algorithm so that invalid data is caught at the boundary before any processing happens.

- Zone Validation Rationale: Allowing arbitrary zone numbers would cause calculation errors or produce unfair default pricing. By placing a While decision loop around the zone entry, the system forces the user to enter a number strictly between 1 and 4.
- Capacity Validation Rationale: Commercial vehicles in Sierra Leone have strict passenger limits. Allowing a passenger count that exceeds vehicle capacity risks severe safety issues and overloading. The validation logic checks passenger count against legal capacity and refuses to calculate a booking until valid numbers are supplied.

6.2 Flowchart & Layout Rationale

I arranged the flowchart logic so that execution arrows flow strictly from top-to-bottom and left-to-right without crossing lines. When an invalid entry occurs, the decision diamond points a reject arrow directly back to the respective input box. This loop structure provides a clear, foolproof visual path that guarantees bad inputs cannot bypass verification steps.

7. Open Source Software & Digital Public Goods

7.1 Open Source Principles

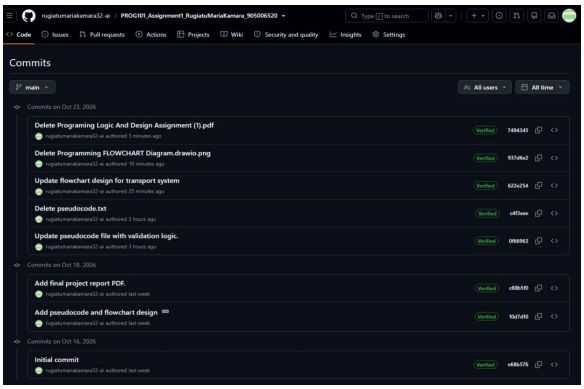
I chose to document this project transparently under an Open Source model. Making programming logic open-source allows other students, local developers, or transport managers in Sierra Leone to take this algorithm, modify it for their own town or route, and re-use it for free without worrying about proprietary software licenses.

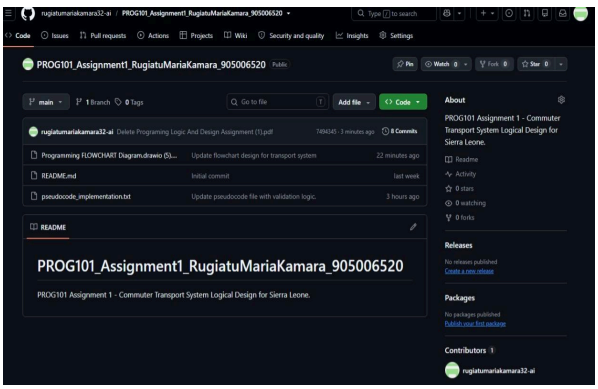
7.2 Version Control & GitHub Deliverables

To track my progress and prove how I built this step-by-step, I set up a public repository on GitHub.

- Repository Name: PROG101_Assignment1_RugiatuMariaKamara_905006520

- RepositoryURL:[rugiatumariakamara32-ai/PROG101_Assignment1_RugiatuMariaKamara_905006520: PROG101 Assignment 1 - Commuter Transport System Logical Design for Sierra Leone.]
- Commit History Summary:
    - Initial project setup and README.md creation
    - Pseudocode and flowchart design
    - Final project report PDF





8. Conclusion & Reflection

Working on this assignment helped me see how programming logic applies directly to real life in Sierra Leone. By taking a simple problem, poda-poda fare confusion, and applying computational thinking, I built a structured algorithm with proper input validation and decision loops. Learning how to document code, draw clean flowcharts, and manage version control on GitHub gives me a strong foundation for future programming modules.

9. References

- Digital Public Goods Alliance. (2025). *Standard for Digital Public Goods*. DPGA Governance Publications.
- Farrell, J. (2018). *Programming Logic and Design, Comprehensive* (9th ed.). Cengage Learning.
- Fullah, E. (2026). *PROG101 Principles of Programming Logic and Design: Individual Assignment Brief*. Department of Information and Communication Technology, Limkokwing University of Creative Technology, Sierra Leone.
- Open Source Initiative. (2026). *The Open Source Definition*. Available online: https://opensource.org/osd
- United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development (Goal 11: Sustainable Cities and Communities)*. New York: UN Department of Economic and Social Affairs.